

Linear Models

1. The Concept of Linear Models

Linear models make a prediction using a linear function of the input features. For regression, the general prediction formula for a linear model looks as follows:

$$\hat{y} = w[0] * x[0] + w[1] * x[1] + \dots + w[p] * x[p] + b$$

- $x[0]$ to $x[p]$: denote the features
- $w \rightarrow$ model coefficients (weights)
- $b \rightarrow$ bias (intercept)
- $\hat{y} \rightarrow$ predicted output

For a dataset with a single feature, this is:

$$\hat{y} = w[0] * x[0] + b$$

Trying to learn the parameters $w[0]$ and b on our one-dimensional wave dataset might lead to the following line (see Figure 5-1):

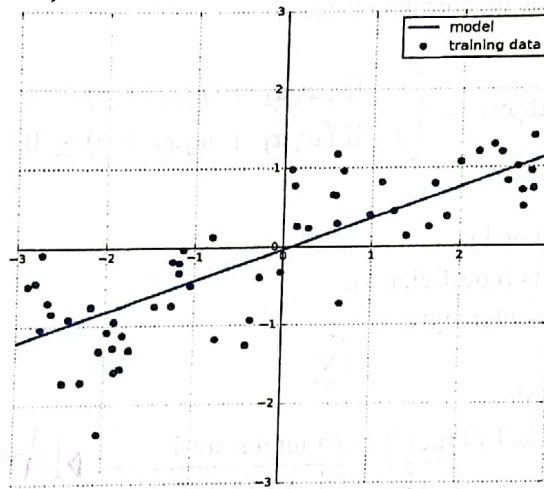


Figure 1-5. Predictions of a linear model on the wave dataset

How w and b are Calculated in Machine Learning (Optimization)

The values for the weights (w) and the bias (b) are not chosen manually; they are learned through a process called **Optimization**.

The Essential Steps for Learning Parameters

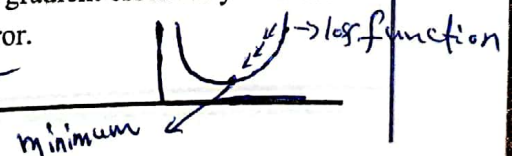
The learning process is driven by iteratively minimizing the error of the model:

1. Defining the Loss Function (Cost Function)

- The **Loss Function** measures the error in the model's predictions. The higher the error, the larger the value of this function.
- The goal of the optimization process is to find the parameters that result in the lowest possible value for the Loss Function.

2. The Optimization Algorithm: Gradient Descent

- The algorithm starts by assigning **random initial values** to w and b .
- It then **iteratively adjusts** w and b in the direction that **reduces the Loss Function**.
- The adjustment is calculated based on the **Gradient**, which is the rate at which the Loss Function changes with respect to each parameter (w and b). The gradient essentially tells the algorithm which direction is "downhill" toward the minimum error.



- By using Gradient Descent, the model moves step-by-step toward the set of parameters (w and b) that minimizes the error, thereby making the decision boundary (the separating line/hyperplane) the best fit for the training data.

Final Result (The Learned Parameters) (w, b)

The final values of w and b are the ones that produce the lowest possible error between the model's predictions and the true targets of the training data, while also taking into account any **Regularization** constraints (like L2 in Ridge Regression) that may have been applied to prevent overfitting.

This example demonstrates how **Linear Models** can be used as a simple classifier (similar to a single-layer perceptron or a simple Logistic Regression) to solve the basic **AND** and **OR** logical functions.

The Linear Model Formula

A linear model for classification works by calculating a weighted sum of the inputs, plus a bias term, and then checking if the result is greater than zero.

$$\text{Output} = \begin{cases} 1 & \text{if } (w_1 x_1 + w_2 x_2 + b) > 0 \\ 0 & \text{if } (w_1 x_1 + w_2 x_2 + b) \leq 0 \end{cases}$$

Where:

- x_1, x_2 : Input features (0 or 1).
- w_1, w_2 : Learned weights (coefficients).
- b : Learned bias term (y-intercept).

1. The AND Function Example

The AND function only outputs 1 (True) if both inputs are 1.

x_1	x_2	y (Target)
0	0	0
0	1	0
1	0	0
1	1	1

Learned Parameters

We can successfully model the AND function using the following parameters:

- Weight $w_1 = 1$
- Weight $w_2 = 1$
- Bias $b = -1.5$

Calculation and Prediction ($w_1 x_1 + w_2 x_2 + b$)

Input (x_1, x_2)	Calculation ($1x_1 + 1x_2 - 1.5$)	Result	Prediction	Correct?
(0, 0)	$1(0) + 1(0) - 1.5 = -1.5$	≤ 0	0	✓
(0, 1)	$1(0) + 1(1) - 1.5 = -0.5$	≤ 0	0	✓
(1, 0)	$1(1) + 1(0) - 1.5 = -0.5$	≤ 0	0	✓
(1, 1)	$1(1) + 1(1) - 1.5 = 0.5$	> 0	1	✓

2. The OR Function Example



The OR function outputs 1 (True) if at least one input is 1.

x1	x2	y (Target)
0	0	0
0	1	1
1	0	1
1	1	1

0
1
1
1

Learned Parameters

We can successfully model the OR function using the following parameters:

- Weight $w_1 = 1$
- Weight $w_2 = 1$
- Bias $b = -0.5$

Calculation and Prediction ($w_1 x_1 + w_2 x_2 + b$)

Input (x_1, x_2)	Calculation ($1x_1 + 1x_2 - 0.5$)	Result	Prediction	Correct?
(0, 0)	$1(0) + 1(0) - 0.5 = -0.5$	≤ 0	0	✓
(0, 1)	$1(0) + 1(1) - 0.5 = 0.5$	> 0	1	✓
(1, 0)	$1(1) + 1(0) - 0.5 = 0.5$	> 0	1	✓
(1, 1)	$1(1) + 1(1) - 0.5 = 1.5$	> 0	1	✓

true = Prediction

اذن النموذج صحيح
نسبة دقة 100%

```
import numpy as np
from sklearn.linear_model import LinearRegression

# المدخلات
X = np.array([[0,0],[0,1],[1,0],[1,1]])

# ===== AND =====
y_and = np.array([0,0,0,1])
model_and = LinearRegression()
model_and.fit(X, y_and)
pred_and = model_and.predict(X)

# تقريب القيم إلى 0 أو 1
pred_and_bin = (pred_and >= 0.5).astype(int)

print("--- AND Function ---")
print("Targets:", y_and)
print("Predictions:", pred_and)
print("Predictions : ", pred_and_bin)
print(f"w1={model_and.coef_[0]:.2f}, w2={model_and.coef_[1]:.2f}, b={model_and.intercept_:.2f}\n")

# ===== OR =====
y_or = np.array([0,1,1,1])
model_or = LinearRegression()
model_or.fit(X, y_or)
pred_or = model_or.predict(X)

# تقريب القيم إلى 0 أو 1
pred_or_bin = (pred_or >= 0.5).astype(int)

print("--- OR Function ---")
print("Targets:", y_or)
print("Predictions:", pred_or)
print("Predictions : ", pred_or_bin)
print(f"w1={model_or.coef_[0]:.2f}, w2={model_or.coef_[1]:.2f}, b={model_or.intercept_:.2f}")

=====
--- AND Function ---
Targets: [0 0 0 1]
Predictions: [-0.25  0.25  0.25  0.75]
Predictions : [0 0 0 1]
w1=0.50, w2=0.50, b=-0.25

--- OR Function ---
Targets: [0 1 1 1]
Predictions: [0.25  0.75  0.75  1.25]
Predictions : [0 1 1 1]
w1=0.50, w2=0.50, b=0.25
```

There are many different linear models for regression. The difference between these models lies in how the model parameters w and b are learned from the training data, and how model complexity can be controlled. We will now take a look at the most popular linear models for regression.

A. Linear regression (OLS ordinary least squares)

Linear regression, or *ordinary least squares* (OLS), is the simplest and most classic linear method for regression. Linear regression finds the parameters w and b that minimize the *mean squared error* between predictions and the true regression targets, y , on the training set.

The mean squared error is the sum of the squared differences between the predictions and the true values. Linear regression has no parameters, which is a benefit, but it also has no way to control model complexity.

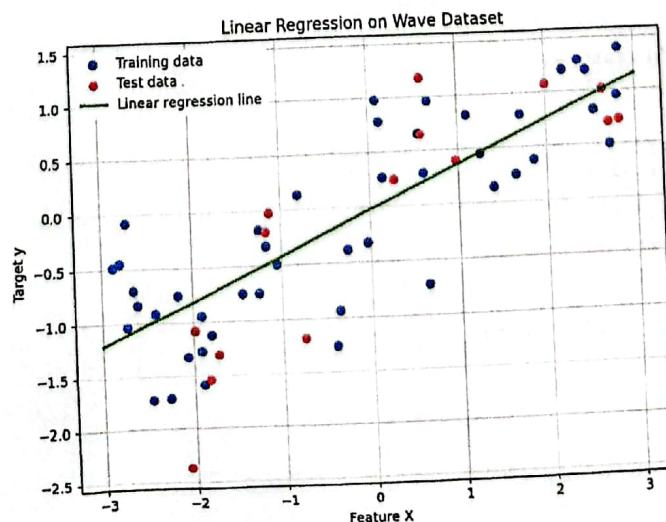
```
import mglearn
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

# إنشاء البيانات (مجموعة الموجة)
X, y = mglearn.datasets.make_wave(n_samples=60)
# تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
# إنشاء نموذج الانحدار الخطي وتدريبه
lr = LinearRegression().fit(X_train, y_train)
# طباعة المعاملات (الميل والتقاطع)
print("Coefficient (w):", lr.coef_)
print("Intercept (b):", lr.intercept_)
# حساب دقة النموذج على التدريب والاختبار
print("Training set score: {:.2f}".format(lr.score(X_train, y_train)))
print("Test set score: {:.2f}".format(lr.score(X_test, y_test)))

# رسم البيانات وخط الانحدار
plt.figure(figsize=(8, 6))
# النقاط الأصلية (البيانات)
plt.scatter(X_train, y_train, color='blue', label="Training data")
plt.scatter(X_test, y_test, color='red', label="Test data")

# خط الانحدار التنبؤي
line_x = np.linspace(-3, 3, 100)
line_y = lr.coef_ * line_x + lr.intercept_
plt.plot(line_x, line_y, color='green', linewidth=2, label="Linear regression line")

# إعدادات الرسم
plt.xlabel("Feature X")
plt.ylabel("Target y")
plt.title("Linear Regression on Wave Dataset")
plt.legend()
plt.grid(True)
plt.show()
```



B. Ridge Regression

Ridge Regression is a **linear regression model** that includes a **regularization term** to prevent overfitting. It uses the **same prediction formula** as Ordinary Least Squares (OLS). However, unlike OLS, Ridge adds a **penalty** on the size of the coefficients (w). The goal is not only to fit the training data well but also to keep the coefficients **as small as possible**. This penalty is called **L2 regularization**. MSE = Mean Squared Error (difference between predictions and actual values)

- α (alpha) = regularization strength (controls how much we penalize large coefficients)

```
import mglearn
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split

# إنشاء بيانات Boston (ممتدة)
X, y = mglearn.datasets.load_extended_boston()

# تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

# وتدريبه Ridge إنشاء نموذج
ridge = Ridge(alpha=1.0).fit(X_train, y_train)

# طباعة نتائج النموذج
print("Training set score: {:.2f}".format(ridge.score(X_train, y_train)))
print("Test set score: {:.2f}".format(ridge.score(X_test, y_test)))
```

C. Lasso

As with ridge regression, using the lasso also **restricts coefficients** to be close to zero, but in a slightly different way, called L1 regularization. The consequence of L1 regularization is that when using the lasso, some coefficients are **exactly zero**. This means some features are entirely ignored by the model. This can be seen as a form of automatic **feature selection**. Having some coefficients be exactly zero often makes a model easier to interpret, and can reveal the most important features of your model.

Let's apply the lasso to the extended Boston Housing dataset:

```
import mglearn
import numpy as np
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split

# Boston الموسعة (من mglearn) تحميل بيانات
X, y = mglearn.datasets.load_extended_boston()

# تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

# وتدريبه Lasso إنشاء نموذج
lasso = Lasso(alpha=0.01, max_iter=100000).fit(X_train, y_train)

# طباعة النتائج
print("Training set score: {:.2f}".format(lasso.score(X_train, y_train)))
print("Test set score: {:.2f}".format(lasso.score(X_test, y_test)))
print("Number of features used:", np.sum(lasso.coef_ != 0))
```